RAG-BASED CHAT SYSTEM

Project Report

A Retrieval-Augmented Generation chat application combining semantic document retrieval, persistent conversational memory with session resume, and large language model generation.

Submitted By	Laiba Murtaza
Project Title	RAG-Based Chat System
Repository	github.com/LaibaMurtaza-21/rag-chat-system
Submission Date	14th July 2026

Table of Contents

1. Introduction

Artificial Intelligence chatbots are becoming increasingly popular for answering user queries. However, traditional Large Language Models (LLMs) depend only on their pre-trained knowledge and cannot access custom documents unless additional mechanisms are provided.

Retrieval-Augmented Generation (RAG) solves this limitation by retrieving relevant information from an external knowledge base before generating a response. This project implements a complete RAG-based chat system capable of answering questions using both stored documents and the LLM's general knowledge, while maintaining persistent conversation history that the assistant actively uses — not just stores — across and within sessions.

2. Project Objective

The objective of this project is to develop a Retrieval-Augmented Generation (RAG) chat application that:

- Provides an interactive, browser-based chat interface.
- Retrieves relevant information from a set of stored documents.
- Uses a Large Language Model to generate context-aware responses.
- Stores chat history permanently across sessions.
- Reuses previous conversation turns as active context for follow-up questions.
- Allows a user to resume any past session and continue it with full context.
- Demonstrates the complete, end-to-end RAG workflow.

3. Problem Statement

Traditional AI chatbots generate responses only from the knowledge learned during training. They cannot answer questions related to custom or private documents unless that information is manually included in the prompt, which does not scale, and they typically forget everything once a conversation ends.

This project addresses both limitations: a Retrieval-Augmented Generation pipeline searches relevant documents and injects the most relevant content into the prompt, while a persistent SQLite-backed memory layer feeds recent conversation turns back into every new prompt so the assistant can recall facts stated earlier — even after the session is closed and resumed later.

4. Project Overview

The system consists of two major components:

Frontend

- Streamlit chat interface with session resume controls

Backend

- FastAPI REST API
- ChromaDB vector database
- SQLite database for persistent chat history
- Sentence-Transformers embedding model
- Groq LLM API

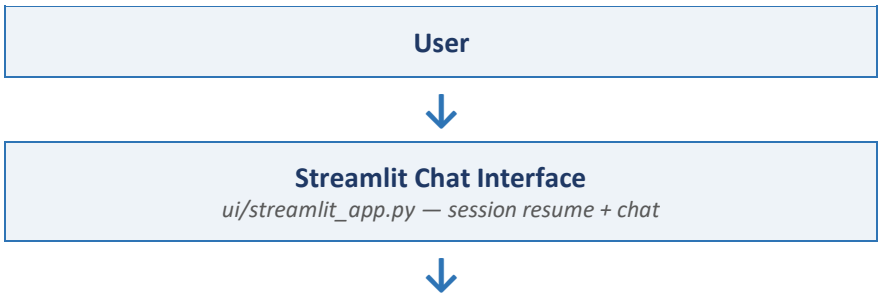
The user interacts with the Streamlit interface. The query is sent to FastAPI, where the RAG pipeline retrieves relevant documents, loads previous conversation history for that session, constructs a combined prompt, sends it to the Groq LLM, and returns the generated response.

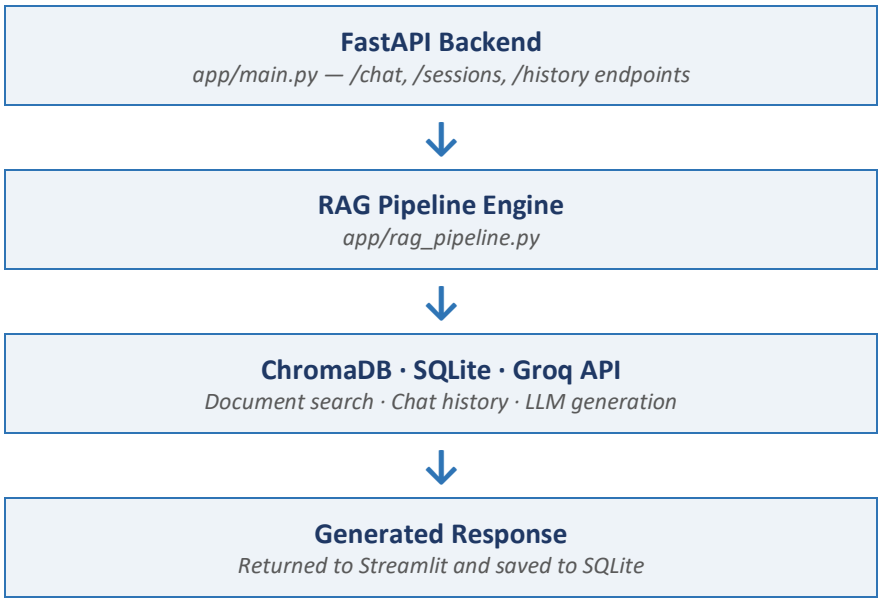
5. Technologies Used

Component	Technology
Programming Language	Python
Frontend	Streamlit
Backend	FastAPI
Vector Database	ChromaDB
Embedding Model	sentence-transformers (all-MiniLM-L6-v2)
Chat Memory	SQLite
Large Language Model	Groq API
LLM Model	llama-3.3-70b-versatile
Environment Variables	python-dotenv

6. System Architecture

The diagram below illustrates the high-level flow of data through the system, from user input to generated response.




FastAPI Backend
app/main.py — /chat, /sessions, /history endpoints
RAG Pipeline Engine
app/rag_pipeline.py
ChromaDB · SQLite · Groq API
Document search · Chat history · LLM generation
Generated Response
Returned to Streamlit and saved to SQLite

7. Project Structure

```
rag-chat-system/
├── app/
│   ├── config.py          # Central configuration (paths, model names, chunk
settings)
│   ├── embeddings.py      # Converts text into vector embeddings
│   ├── vector_store.py    # ChromaDB wrapper for storing/retrieving chunks
│   ├── history.py         # SQLite wrapper for persistent chat history
│   ├── llm.py             # Groq API client for generating responses
│   ├── rag_pipeline.py    # Combines retrieval + history + generation
│   └── main.py            # FastAPI backend: /chat, /sessions, /history
endpoints
├── ui/
│   └── streamlit_app.py   # Chat interface with session resume support
├── data/                  # Source documents to be ingested (.txt files)
├── storage/               # Auto-generated: ChromaDB + SQLite database files
├── screenshots/           # Demonstration screenshots
├── ingest.py              # Script to chunk and embed documents
├── requirements.txt
├── .env                   # API keys (excluded from version control)
├── .gitignore
└── README.md
```

8. Working of the RAG Pipeline

The implemented RAG pipeline consists of four major stages.

Step 1 — Document Ingestion

Documents stored inside the data/ folder are read and divided into smaller, overlapping chunks. Each chunk is converted into vector embeddings using the all-MiniLM-L6-v2 embedding model. The embeddings are stored inside ChromaDB.

Step 2 — Retrieval

When the user submits a question, it is converted into an embedding, ChromaDB performs a similarity search, and the most relevant document chunks are retrieved.

Step 3 — Augmentation

The retrieved document chunks are combined with the most recent turns of chat history for the active session and the current user query to create a complete prompt. If no relevant documents are retrieved, the system allows the LLM to answer using its own general knowledge, while still using the conversation history.

Step 4 — Generation

The final prompt is sent to the Groq API using the llama-3.3-70b-versatile model. The generated answer is returned to the user and saved into SQLite, ready to be retrieved for the next turn or a future resumed session.

9. Implementation Details

Streamlit Interface

Provides the interactive chat interface. Responsibilities:

- Accept user questions.
- Display the ongoing conversation.
- Send requests to the FastAPI backend.
- Display AI-generated responses.
- List and resume past sessions via the sidebar.

FastAPI Backend

Acts as the communication layer. Responsibilities:

- Receive user messages.
- Invoke the RAG pipeline.

- Return generated responses.
- Expose session listing and history retrieval endpoints.

Vector Database (ChromaDB)

Stores document embeddings. Responsibilities:

- Store embeddings persistently.
- Perform semantic similarity search.
- Retrieve relevant document chunks.

Embedding Model

The project uses sentence-transformers/all-MiniLM-L6-v2 to:

- Convert documents into embeddings.
- Convert user questions into embeddings.

Large Language Model

The project uses the Groq API with the llama-3.3-70b-versatile model to:

- Generate context-aware responses.
- Answer using retrieved information, conversation history, and general knowledge.

10. Chat Memory Management

Persistent chat history is implemented using SQLite. Each conversation stores a session ID, the user message, the assistant response, and a timestamp. Sessions are identified by a UUID generated per browser session.

On every new message, the pipeline calls chat_history.get_history(session_id) to retrieve the most recent messages for that session, which are folded into the prompt sent to the LLM. This means the assistant does not merely log conversations — it actively uses them as context, allowing it to answer follow-up questions such as recalling a name or fact stated earlier in the same conversation.

When a user resumes a previous session from the sidebar, the full stored history for that session is reloaded into the interface and continues to be used as context for any further questions.

11. Data Storage and Retrieval

The project uses two storage mechanisms:

ChromaDB

Stores vector embeddings for document retrieval.

SQLite

Stores user messages, assistant responses, session IDs, and full conversation history. This satisfies the project requirement of persistent, reusable chat memory.

12. API Endpoints

Endpoint	Description
POST /chat	Send a message and session ID; returns a RAG-generated answer.
GET /sessions	Lists all past session IDs, most recent first.
GET /history/{session_id}	Retrieves the full stored message history for a session.
GET /	Health check confirming the API is running.

13. How to Run the Project

1. Clone the repository

```
git clone https://github.com/LaibaMurtaza-21/rag-chat-system.git
cd rag-chat-system
```

2. Create a virtual environment

```
python -m venv venv
venv\Scripts\activate        # Windows
source venv/bin/activate     # Mac/Linux
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Configure the API key

Create a .env file in the project root:

```
GROQ_API_KEY=your_groq_api_key
```

5. Ingest documents

```
python ingest.py
```

6. Start the backend

```
uvicorn app.main:app --reload
```

7. Start the frontend

```
streamlit run ui/streamlit_app.py
```

14. Demonstration — Chat Memory in Action

The screenshot below demonstrates persistent memory working correctly within a live session. The user states their name, asks an unrelated follow-up question, and the assistant correctly recalls the name several turns later — confirming that conversation history is being actively read back into the prompt on every turn, not merely stored.

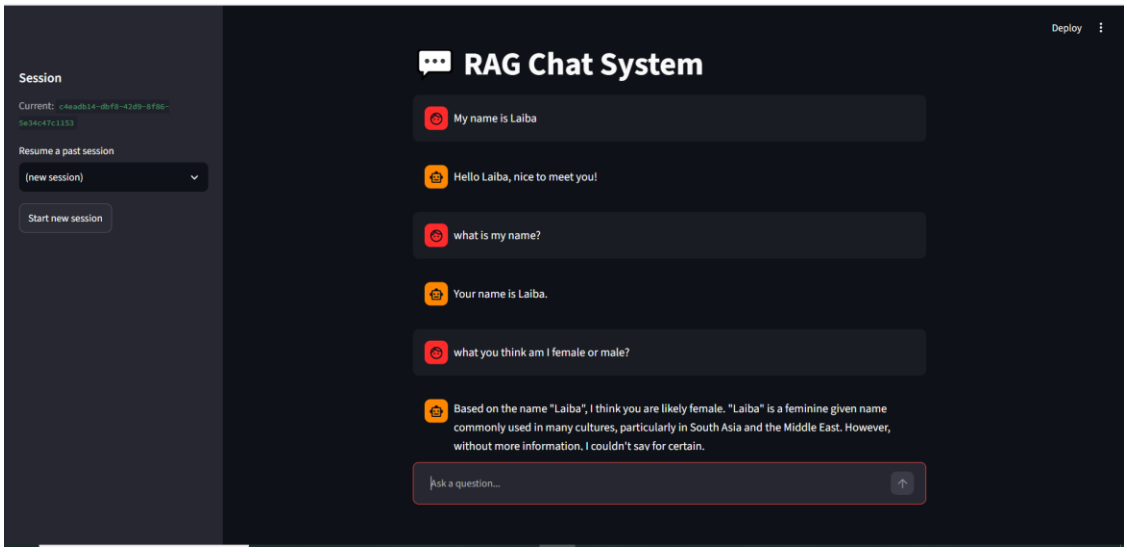


Figure 1: The assistant recalling a name stated earlier in the same session.

15. Debugging and Issue Resolution

During development and testing, several defects surfaced in the persistence and session-resume logic. Each was diagnosed from the FastAPI traceback and resolved directly in the affected module. These are documented below as evidence of the debugging process.

Issue 1 — SQLite directory not created

Cause: sqlite3.connect() failed because the target storage directory did not exist yet on first run.

Fix: history.py now calls os.makedirs(os.path.dirname(self.db_path), exist_ok=True) before connecting, guaranteeing the storage/ directory exists.

Issue 2 — sqlite3.OperationalError: misuse of aggregate: MAX()

Cause: list_sessions() used SELECT DISTINCT session_id ... ORDER BY MAX(timestamp), but SQLite does not allow an aggregate function in ORDER BY without a corresponding GROUP BY.

Fix: Replaced DISTINCT with GROUP BY session_id, which both deduplicates sessions and makes MAX(timestamp) a valid aggregate to order by.

Issue 3 — AttributeError: 'ChatHistory' object has no attribute 'list_sessions'

Cause: The list_sessions method definition was accidentally left at column 0 (no indentation), so Python parsed it as a standalone function outside the ChatHistory class rather than a method.

Fix: Re-indented the method body to align with the class's other methods, correctly attaching it to ChatHistory.

Issue 4 — Assistant did not use earlier messages in the same session

Cause: get_history() queried ORDER BY id ASC LIMIT ?, which returns the oldest messages in a session rather than the most recent ones — so once a session grew past a few turns, only the earliest exchanges were ever sent back to the LLM.

Fix: Changed the query to ORDER BY id DESC LIMIT ? to fetch the most recent messages, then reversed the result in Python to restore chronological order before building the prompt. Verified by asking the assistant to recall a name stated earlier in the session (see Figure 1).

16. Example Workflow

User Question

"Where is the Eiffel Tower?"

System Process

- Convert the question into an embedding.
- Search ChromaDB for relevant chunks.
- Retrieve the matching document.
- Retrieve previous conversation history for the session.
- Build the combined prompt.
- Send the prompt to Groq.
- Generate the response.
- Save the conversation turn to SQLite.

Response

"The Eiffel Tower is located in Paris, France."

17. Challenges Faced

During implementation, the following challenges were encountered:

- Configuring the Groq API.
- Managing environment variables securely.
- Integrating FastAPI with Streamlit.
- Implementing persistent chat history that is actually used as context, not just logged.
- Diagnosing SQLite aggregate-function and indentation errors from raw tracebacks.
- Handling document ingestion and vector storage.
- Managing project structure and GitHub deployment.

These challenges were resolved through systematic debugging: reading each traceback, isolating the failing line, and verifying the fix by reproducing the original test case.

18. Future Improvements

- Support for PDF and DOCX documents.
- User authentication.
- Cross-session memory (recalling facts from a different past session).
- Conversation search.
- Source citation for retrieved answers.
- Docker containerization.
- Cloud deployment.
- Streaming responses.
- Multi-user support.

19. Conclusion

This project successfully demonstrates the implementation of a Retrieval-Augmented Generation (RAG) chat system using modern AI technologies. The application combines semantic document retrieval, persistent and actively-used conversation memory, and a Large Language Model to generate context-aware responses.

The project fulfills the required objectives by integrating a Streamlit frontend, FastAPI backend, ChromaDB vector database, SQLite-based chat history with verified session memory and resume capability, and the Groq LLM API into a complete, tested, end-to-end system.

20. References

- Groq API Documentation
- FastAPI Documentation
- Streamlit Documentation
- ChromaDB Documentation
- Sentence Transformers Documentation
- SQLite Documentation